MacFAST – Scalable Adaptive Testing Platform

Group 10

COMPSCI 4ZP6 Capstone Project

Dr. Mehdi Moradi

Oct 10th, 2025

Full Name	**Email Address (McMaster)**	**Student Number**
Dai Ishida	ishidad@mcmaster.ca	400435830
Mohib Ur Rehman Siddiqui	siddim93@mcmaster.ca	400415895
Jack Fountain	fountj1@mcmaster.ca	400333572
Laeek Ahmed Shaikh	shaikl3@mcmaster.ca	400358738
Raj Rehman	moham50@mcmaster.ca	400359821
Peter Wardell	wardelp@mcmaster.ca	400399034
Jwalit Bhavinkumar Miniwala	miniwalj@mcmaster.ca	400357388

Contents

Version 3

Glossary 3

Contribution History 3

Project Purpose 3

1 Software Requirements Specification (SRS) 4

1.1 Clients & Stakeholders 4

1.2 Project Constraints 5

1.3 Functional Requirements 5

1.4 Data & Metrics 6

1.5 Non-functional Requirements 6

1.6 Legal Requirements 8

1.7 Predicted Risks & Legal Issues 9

2 Project Development Plan 9

2.1 Team Meeting & Communication Plan 9

2.2 Team Member Roles 10

2.3 Workflow Plan 12

2.4 Proof of Concept Demonstration Plan 13

2.5 Technology 13

2.6 Project Schedule 15

Version

Version number	Date	Description	Author
0	Oct 10, 2025	Initial draft	Everyone

Glossary

Abbreviations:

- A2L: Avenue to Learn
- FS: File system (on a machine)
- IRT: Item Response Theory (assessment modeling)
- JWT: JSON Web Tokens
- MVP: Minimum Viable Product
- OIDC: OpenID Connect (Authentication Protocol)
- PR: Pull Request
- SEO: Search Engine Optimization
- SSO: Single Sign-On
- SSR: Server-Side Rendering
- UTS: University Technology Services
- VM: UTS Virtual machine(s)

All code and artifact naming conventions:

- snake_case for files
- CamelCase for classes
- lowercase_with_underscores for variables/methods.

Definitions:

- Adaptive Testing: The algorithmic process by which the system delivers individualized questions based on past performance

Contribution History

Authors	**Sections**
Laeek Ahmed Shaikh	Everything before section 1 + 2.3
Mohib Ur Rehman Siddiqui	1.1, 1.2
Peter Wardell	1.3, 1.4
Raj Rehman	1.5.1, 1.5.2, 1.5.3, 1.5.4, 1.5.5, 1.6.1, 1.6.2
Dai Ishida	2.1, 2.2
Jack Fountain	2.4
Jwalit Bhavinkumar Miniwala	2.5, 2.6

Project Purpose

The purpose of this project is to develop Mac-FAST, a scalable, web-based platform for practice testing, which is an effective way to study and promote long-term retention of material.

This approach also aligns with the preferred study strategy of students who have taken introductory chemistry and kinesiology courses (based on internal research by Dr. Greenberg). Mac-FAST will support practice using a sizable archive of prior multiple-choice tests and exams.

In addition, Mac-FAST will incorporate adaptive testing. Adaptive testing uses statistical data to deliver questions that are individualized for each student: answering a question correctly results in the next question being more difficult; conversely, answering a question incorrectly results in the next question being easier.

In effect, each student would receive a personalized set of practice questions, tailored to their individual needs, to maximize their preparation for assessments and improve their metacognitive skills. Our tool will also show students a realistic picture of their progress and performance, so that they are challenged at just the right level.

1 Software Requirements Specification (SRS)

1.1 Clients & Stakeholders

The client for this project is the Department of Chemistry and Chemical Biology, and the Department of Kinesiology, McMaster University.

Primary Stakeholders:

- **Students (End Users):** Undergraduate students enrolled in introductory chemistry and kinesiology courses who will use Mac-FAST for practice testing and self-assessment.
- **Instructors and TAs:** Responsible for uploading questions, configuring assessments, and reviewing student progress.
- **Supervising Team:** Provide academic and technical oversight, approve design decisions, and ensure alignment with course objectives and university standards.
- **McMaster UTS:** Manage server hosting, security compliance, and Single Sign-On.
- **Development Team:** COMPSCI4ZP6 Group 10; responsible for system design, development, testing, and documentation of the Mac-FAST platform.
- **Capstone Instructor/TAs:** Dr. Moradi, Sahib Khokhar, Amirhossein Sabour.

Secondary Stakeholders:

- People attending/delivering academic help sessions may use the platform for preparation/testing.

1.2 Project Constraints

1. **Hosting Environment:** The system must be hosted on university managed on-premises servers. Use of cloud platforms is not permitted.
2. **Authentication:** User login must integrate with the university's Single Sign-On (SSO) solution.
3. **Application Type:** Mac-FAST must be a web-based application accessible through standard desktop and mobile browsers.
4. **Technology:** It should have a model-view-controller (MVC) structure, allowing for server-side rendering. This will allow to separate back end from the front end and design future-proof tool
5. **Performance and Scalability:** The system must sustain up to 1500 concurrent student users, particularly during exam periods.
6. **Adaptive Testing Model:** The system will incorporate a fully functional adaptive testing mechanism that dynamically adjusts question difficulty based on each student's performance. The adaptive engine will use Item Response Theory (IRT) to estimate a learner's ability level.
7. **Design and Branding:** All visual design must comply with the university's official branding guidelines, including approved color palette, typography, and layout standards. The interface should visibly reflect the institution's identity while maintaining accessibility best practices.

1.3 Functional Requirements

1.3.1 Priority 0 (Minimum Viable Product)

- Use McMaster's SSO for authentication/student login
- An adaptive testing framework that will consider the student's "ability score", calculated difficulty of the question, and what topic they are currently studying to determine which question will be asked next.
- Support ~7000 total users and at least a significant fraction of concurrent users
- Ability to upload test banks/ questions in multiple formats containing multiple types of questions from multiple sources and parse the questions from them
    - CSVs from A2L quiz exports
    - Word Documents
    - HTML files from Scantron
    - Crowdmark exports
    - Ability to parse LaTeX within each of the above
- The ability to set up classes and assign students to those classes. Students will only then access the tests for that class.
- Students are only able to view their own information, ability score, and progress.
    - Comparisons to the class average in terms of correctness, and time taken on questions

    - Data on how much of a certain topic has been covered
    - Data on which topics the student is struggling with
- Instructors can view each student's information, ability score, and progress as well as comment on questions with other instructors
- The ability to recognize "bad" questions through flags and analytics
- A well navigable UI that follows the McMaster colour and design guidelines

1.3.2 Priority 1 (Next Feature Set)

- The ability for instructors to modify individual questions in a specific test bank.
- The ability to update the difficulty rating of each question based on aggregate student answers.
- Ability to bookmark questions to revisit instead of guessing as well as a section to practice questions the user often messes up based on unit/ type of question.

1.3.3 Priority 2 (Non-critical Features)

- An AI chatbot specific to each course that students can talk to for more clarification
- Support for more courses, in different departments along with the different types of questions that they will bring.

1.4 Data & Metrics

Data is not applicable to this project (In the way it would be for a machine learning project)
Metrics

- Accessibility (Server uptime and page load times)
    - We can use load testing software to determine if the max load of our server meets the requirements. Another option is to open a beta version early for students to access and test it that way.
- Adaptive testing
    - We should see that students are being asked questions that match their "ability score". If the adaptive testing is working, all students should be getting a similar number of questions correct, while their ability score shows their actual performance.
- Site usability (Uploading test banks, completing tests)
    - Successful uploading, parsing and displaying of test banks will be our metric of success. Early student/ instructor feedback on beta versions can help us with this as well.

1.5 Non-functional Requirements

1.5.1 Look and Feel Requirements

- The interface must maintain a consistent, academic-style design across all pages.
    - Use McMaster-inspired color palette and theme for visual identity.
    - Ensure uniform navigation menus, page layouts, and headers across all sections.

- The platform shall be simple and distraction-free to promote focused studying.
  - Minimalist layout with clear question areas and progress indicators.
  - Light animations and progress indicators will provide feedback without cluttering the page.
- The interface must be responsive across multiple devices.
  - Support for desktop, tablet, and mobile browsers.
  - Content automatically adjusts to fit screen size and resolution.
- Accessibility standards shall be met to ensure clear and readable design.
  - Fonts and colors should meet readability standards and offer good contrast.
  - The design will avoid colors choices that make it hard for color-blind users to distinguish elements of the site.

1.5.2 Usability and Humanity Requirements

- The system should be intuitive enough so that students can use it without requiring training or manuals.
  - Students should be able to begin practice testing without tutorials.
  - A brief optional onboarding walkthrough may be provided.
- Consistency and predictability shall be maintained across all modules, by reusing similar buttons, icons, and interaction patterns.
- The system shall provide continuous feedback to users.
  - Correct/incorrect responses shown immediately after submission.
  - Progress bars and completion percentages displayed throughout.
- Error prevention and recovery shall be implemented.
  - Warn users before exiting a quiz in progress.
  - Auto-save ongoing sessions periodically.
- Accessibility and inclusion requirements must be met, aiming to follow common accessibility standards wherever feasible within the project scope.
- A help section and FAQ shall be available within the platform, which will include troubleshooting steps and relevant contact information.

1.5.3 Performance and Speed Requirements

- The system shall perform efficiently under expected load.
  - Page loads and navigation within 2 seconds under normal conditions.
  - Question transition within 1 second to maintain flow of study sessions.
- Scalability requirements:
  - Support up to 1,500 concurrent users without significant performance degradation.
  - Allow horizontal scaling for higher demand periods (e.g. exams).
- The platform should remain reliable and accessible for users throughout the semester.
- Commonly accessed data such as question sets and progress reports will be cached to improve site performance.
- The system should be able to recover smoothly after unexpected interruptions or downtime.

1.5.4 Security and Privacy Requirements

- All user data shall be protected following McMaster University security standards.
  - Authentication will be handled through McMaster's SSO using OAuth 2.0 with OIDC, so no passwords will be stored or processed by Mac-FAST.
- All server-client data transmission will be encrypted using HTTPS/TLS.
- Implement authentication and authorization mechanisms.
  - Role-based access: students, instructors, and admins with distinct permissions.
- Privacy protection requirements:
  - All data handling will comply with McMaster University's privacy and data governance policies.
  - Student data will not be shared outside authorized academic roles.
  - Only Instructors, TAs and other authorized individuals will have broader access to site backend and student performance data for course management and evaluation purposes.
- Session management will include automatic timeout after periods of inactivity (say 30 minutes), using secure cookies to protect user sessions.
- The system will maintain audit logs of key actions like logins, quiz submissions, and administrative changes.
- Encrypted backups of databases and important assets will be created to ensure data safety and recovery.

Note: For future maintenance (after completion of capstone), all open-source libraries and dependencies will have to be reviewed and updated regularly to ensure continued protection against any security vulnerabilities.

1.6 Legal Requirements

- Compliance with institutional and provincial regulations:
  - Adhere to McMaster University IT and data-governance policies.
  - Comply with Canadian privacy laws (like PIPEDA, FIPPA).
- Copyright and content ownership:
  - All question content must be instructor approved, or institution owned.
  - External material must respect copyright and licensing laws.
- Terms of use and consent:
  - Display privacy policy and user consent upon first login.
  - Outline data retention and removal processes clearly.
- All external libraries and third-party tools used must comply with McMaster University's software usage and licensing policies.

1.7 Predicted Risks & Legal Issues

1.7.1 Technical Risks and Challenges

- **Test Bank Format Variability:**
  There is no consistent format for instructor test banks. Some use Word, while others use Excel, PDF, or CSV. This may cause parsing errors or require extra work during preprocessing.
- **Scalability and Load Handling:**
  With up to 7,000 users of the site, peak usage instances (like before exams) may lead to slower response times or temporary downtime. To minimize this risk, we will conduct early load testing, implement caching, and use container-based scaling to handle increased traffic efficiently.
- **Third-Party Tool Restrictions:**
  McMaster's security and privacy policies may restrict use of certain third-party tools or services. This will require us to build in-house alternatives, which may take longer to develop. SSO and other integrations with McMaster's internal IT infrastructure may also take longer due to dependency on McMaster UTS.

1.7.2 Future Maintenance Post-Handover

After the capstone project is completed and handed over, all the ongoing updates, bug fixes, and future maintenance will have to be handled by the Department of Chemistry and Chemical Biology, Department of Kinesiology, McMaster University, and their supporting technical staff who work part-time. To support a smooth transition, we will provide them with detailed documentation and will offer assistance whenever possible post the project handover. Future updates may also involve integrating new courses, updating existing codebase, or incorporating new tools/frameworks to meet the changing academic and technical requirements.

2 Project Development Plan

2.1 Team Meeting & Communication Plan

Our team's main communication platforms will be Microsoft Teams. We have two main channels:
1. A channel for all team members to discuss project schedules and updates.
2. A channel for all team members, Dr. Greenberg, Dr. Nederveen, and Benjamin, who is an engineer acting as an advisor, to receive feedback on the project.
For document sharing, we will use Microsoft SharePoint through our Teams channel. Throughout the fall semester, we plan to have weekly meetings at the following times:
- Monday from 14:30 to 15:30
- Tuesday from 15:30 to 16:30
- Thursday from 20:00 to 20:50
Meeting dates and time may be changed in the winter semester depending on our supervisor's

schedule and our class schedules. Meetings will be held in person and on Teams in case of member inability to join in person. Our Monday meetings will be focused on collaborative work to further advance our goals. Tuesday meetings will serve as progress updates with Dr. Greenberg and Dr. Nederveen to ensure that we are all on track and meet the project requirements. Thursday meetings will be technical sessions with Benjamin to address project matters. All online meetings are recorded, and all meeting notes will be shared on Teams.
For project management, we will use GitHub issues. Each issue will correspond to specific functional or non-functional requirements and will include a clear description of the tasks as well as assigned team members.

2.2 Team Member Roles

Role	Primary	Secondary	Responsibilities
Project Manager	Peter	Jwalit, Mohib	Oversees project timelines and ensures that all requirements are delivered on schedule. Coordinates effectively between developers and stakeholders to maintain alignment in goals, deliverables, and priorities.
Infrastructure	Laeek	Dai, Raj	Responsible for implementing and managing CI/CD pipelines, designing system architecture, and ensuring system monitoring and scalability to support reliable operations.
Backend	Dai, Jack	Peter	Manages database design and maintenance, API development, data parsing, and system logic implementation. Oversees the development of the admin panel and logging mechanisms to ensure efficient backend functionality.
Frontend	Mohib, Raj	Laeek	Focuses on the development and design of the user interface (UI) and user experience (UX). Ensures that the application's frontend is intuitive, responsive, and aligned with the user needs.
Security Engineer and Analyst	Jwalit	Jack	Ensures the integrity and confidentiality of the application data. Designs and implements

			security measures, identifies potential vulnerability, and provides mitigation strategies to safeguard the system against threats.

Functional Requirements	Primary Developer(s)	Secondary Developer(s)
Use McMaster's SSO for authentication/student login (P0)	Jwalit	Jack
An adaptive testing framework that will consider the student's "ability score", calculated difficulty of the question, and what topic they are currently studying to determine which question will be asked next. (P0)	Dai, Jack	Peter
Support ~7000 total users and at least a significant fraction of concurrent users. (P0)	All	
Ability to upload test banks/ questions in multiple formats containing multiple types of questions from multiple sources and parse the questions from them. CSVs from A2L quiz exports, Word Documents, HTML files from Scantron, Crowdmark exports. (P0)	Dai, Jack	Peter
The ability to set up classes and assign students to those classes. Students will only then access the tests for that class. (P0)	Jwalit	Jack
Students are only able to view their own information, ability score, and progress. Comparisons to the class average in terms of correctness, and time taken on questions. Data on how much of a certain topic has been covered. Data on which topics the student is struggling with. (P0)	Mohib, Raj	Laeek
Instructors can view each student's information, ability score, and progress. (P0)	Jwalit	Jack
The ability to recognize "bad" questions through question flagging and analytics. (P0)	Mohib, Raj	Laeek
A well navigable UI that follows the McMaster colour and design guidelines. (P0)	Mohib, Raj	Laeek
The ability for instructors to modify individual questions in a specific test bank. (P1)	Jwalit	Jack
The ability to update the difficulty rating of each question based on aggregate student answers. (P1)	Dai, Jack	Peter
Ability to bookmark questions to revisit instead of guessing as well as a section to practice questions the user often messes up based on unit/ type of question. (P1)	Mohib, Laeek	Jack

Support for more courses, in different departments along with the different types of questions that they will bring. (P2)	Laeek, Jack	Peter
An AI chatbot specific to each course that students can talk to for more clarification. (P2)	Dai	Peter

Non-Functional Requirements	Primary Developer(s)	Secondary Developer(s)
Look and Feel Requirements	Mohib, Laeek	Raj
Usability and Humanity Requirements	Mohib, Raj	Laeek
Performance and Speed Requirements	Dai	Peter
Security and Privacy Requirements	Jwalit	Jack
Legal Requirements	Jwalit	Jack

2.3 Workflow Plan

2.3.1 Version control

- Repo structure: Separate/frontend, /backend, /infrastructure, /docs repository
- Branching model: main (protected, release-ready), feature/{scope}-{issue#}, bugfix/{issue#}, hotfix/{issue#}.
- Create a tag/release to test in the dev env and then merge to main after approval
- Pull requests: all changes via PR into dev; hotfix PRs can target main with immediate patch releases. Require:
  - Passing CI (build, tests, lint, type-check).
  - 1–2 reviewer approvals
  - Conventional Commits in titles (feat:, fix:, refactor:, docs:, test:).
  - PR template including scope, linked issues, test evidence

2.3.2 Issue management

- GitHub Issues with labels (type: feature/bug/chore/docs, priority: P0–P4, area: frontend/backend/data/devops), milestones aligned to sprints/releases, assignees, estimates, and acceptance criteria.
- Reviews/quality gates: Protected branches, required status checks, branch policies (no direct pushes to main), required linked issue per PR.

2.3.3 Agile process (scrum/sprints)

- Tooling: GitHub Projects (Kanban/sprints) for sprint boards, backlog, and road mapping; we will not use Jira.
- 2-week sprints with planning, weekly async standups (Online on teams or in person on Tuesdays)
- Board columns: Backlog → Ready → In Progress → In Review → Done

- Roles: Product/Project Manager (board, priorities, scope)

2.3.4 Data storage

- Application data: SQL database self-hosted on UTS server for users, sessions, question bank, test results, and adaptive testing parameters
- UTS server filesystem storage for media/assets.
- Secrets/compliance: .env files local only; production secrets in GitHub Environments

2.3.5 Tools/methods mapped to requirements and performance metrics

- Please see section 2.5 Technology where each requirement and performance metric is directly linked to the specific technology or tool used to achieve

2.4 Proof of Concept Demonstration Plan

For the proof-of-concept demonstration, we will be expected to meet all P0 requirements. This includes a basic Next.JS website with a focus on functionality and minimal styling (that follows McMaster's brand guidelines) with authentication protected views. This website will focus on the student's view where they can practice questions and view their personal analytics compared to the class. A primitive Django backend will also be setup which will handle the more challenging aspects of the project.

To integrate the backend with McMaster's SSO, our first step will be to create a Microsoft 365 sandbox. This environment will serve for both development and for showcasing the implemented authentication. Another foreseeable risk is the parsing of Word documents which are rich text documents, that can have minor formatting mistakes by professors and have entirely different structures per department and even professor. It will be challenging to make the platform flexible enough to adapt parsing to different structures. We will have access to a minimum of two different professors' question banks in Word and csv formats to demonstrate it is possible to parse them.

Through Django's customizable backend coupled with external libraries like django-plotly-dash, we can add another app to our Django project that can directly query our data and display it through Dash graphs. Another option could be to have a separate Streamlit app that calls our APIs through our Django app to map out the data. Finally, a major risk will be handling the number of students that could be using the practice website before midterms and exams. To handle this, a barebones Docker compose setup will be made to showcase that is it scalable, which in actual production can be thrown onto Kubernetes.

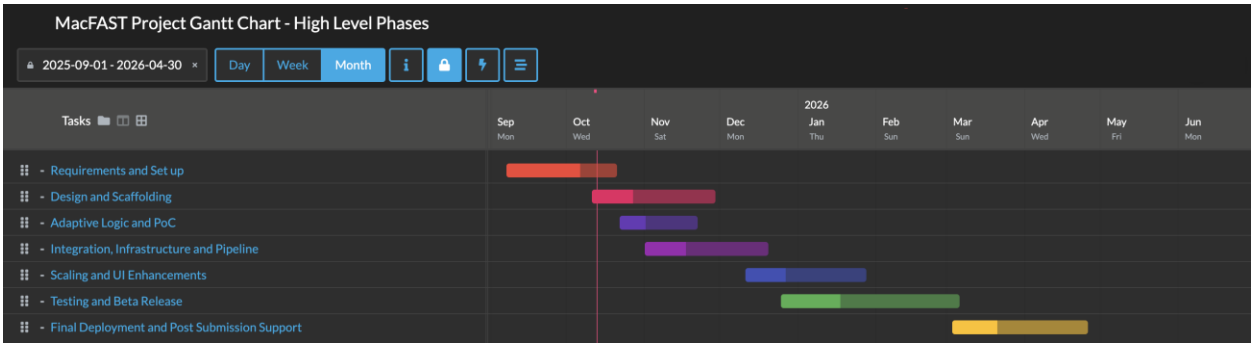
2.5 Technology

The MacFAST system will be developed using a modern web application stack, balancing maintainability, scalability, and compatibility with the wider IT environment at McMaster University.

- **Front-End:** The application will be implemented using Next.js, which is a React-based framework that has SSR capabilities. This will allow for solid performance, optimized SEO, and a modular approach to design. Pages will be rendered in Next.js, while data will be supplied from backend APIs.
- **Back-End:** The application will be implemented in Django (Python), which is a well-established framework for creating API endpoints, user management, and database access. The Django framework is popular, widely adopted, and easy to maintain, which will help ensure it will be viable in the long term. While we considered other lightweight frameworks (i.e., FastAPI) for the backend implementation, we selected Django for its established framework, stability, and familiarity with the current developers on the team.
- **Database:** We will initially use MySQL, given that it is consistent with the previous infrastructure. However, the system will be designed to be database agnostic, in case there is a need to migrate to PostgreSQL or NoSQL in the future. The data ingestion will occur via multiple sources, which may include Word documents, HTML files for URLs, and Avenue CSV exports. All ingested data will undergo a normalization and standardization process before loading it into the database.
- **Hosting and Deployment:**
  - Development instances may be hosted briefly on Vercel to allow for fast deployment.
  - Production deployment will occur on VMs provided by the university, using Docker containers for modularity and portability. We recommend that development, database, and production instances be separate VMs.
  - We will consider load balancing to manage up to ~7,000 concurrent students across multiple classes.
- **Authentication:** McMaster's SSO integration with OAuth 2.0 with OIDC. Pre-existing JWT will be handled using libraries such as Jose in Node.js/Python programming languages.
- **Security:** All student data will remain within McMaster's infrastructure. We will not store sensitive data in cloud-based infrastructure. GitHub will be used for code management, minus student data.
- **Testing:** We will conduct unit and stress testing on adaptive testing logic and on backend API endpoints to adhere to software engineering principles. Continuous integration/continuous deployment (CI/CD) will also be managed using GitHub Actions to trigger automated build and test processes.

This technology stack will provide scalability and maintainability while adhering to McMaster's IT security requirements.

2.6 Project Schedule



Detailed breakdown of the project phases:

Task	Start Date	End Date
Requirements Gathering	Sep 10, 2025	Sep 25, 2025
Project Setup (Repo, Tools, Workflows)	Sep 15, 2025	Sep 30, 2025
Database Schema Design	Oct 01, 2025	Oct 15, 2025
Frontend Scaffold (Next.js)	Oct 01, 2025	Oct 20, 2025
Backend Scaffold (Django)	Oct 01, 2025	Oct 20, 2025
Proof of Concept Demo	Oct 15, 2025	Nov 20, 2025
Design Document	Oct 15, 2025	Jan 23, 2026
Initial Unit Test Environment Setup	Oct 15, 2025	Oct 30, 2025
Ongoing Unit & Integration Testing	Nov 01, 2025	Mar 15, 2026
Authentication Integration (SSO)	Nov 01, 2025	Nov 15, 2025
Data Ingestion Pipeline (Word to CSV to DB)	Nov 05, 2025	Nov 25, 2025
Frontend-Backend Integration	Nov 15, 2025	Nov 30, 2025
Adaptive Testing Algorithm Development	Dec 01, 2025	Dec 15, 2025
Load Balancing & Scaling Setup	Jan 05, 2026	Jan 20, 2026
UI/UX Improvements and Advanced Analytics Dashboard	Jan 10, 2026	Jan 30, 2026
Stress & System Testing (Performance, E2E)	Jan 15, 2026	Mar 10, 2026
Comprehensive Testing (Final Phase)	Feb 01, 2026	Feb 15, 2026
Beta Release Deployment	Feb 16, 2026	Feb 28, 2026
Performance Optimization & Bug Fixing	Mar 01, 2026	Mar 15, 2026
Final Presentation & Demo Preparation	Mar 01, 2026	Mar 25, 2026
Final Deployment & Submission	Mar 25, 2026	Apr 01, 2026
Post-Submission Support & Wrap-up	Apr 01, 2026	Apr 10, 2026